TrackServ Frontend Development Notes

1. Project overview

TrackServ is a civic-service platform that connects residents with municipal service providers. The frontend is split into three role-based applications:

1. **TrackServ citizen application** (`trackserv/`) - residents register, report municipal problems, monitor progress, participate in the community, and manage their profiles.
2. **TrackServ admin dashboard** (`admin/`) - municipal administrators monitor incoming reports, inspect locations, assign work, and analyse service-delivery performance.
3. **TrackStaff portal** (`TrackStaff/servers/`) - field staff view assigned work on a map, attend to issues, update progress, upload evidence, and manage their staff profile.

All three applications use the same Supabase project. This lets a report created by a citizen become visible to administrators and field staff without a separate frontend-to-frontend integration.

2. Technology stack

Core frontend technologies

- **React 19**: component-based UI, local state, effects, and reusable page components.
- **JavaScript/JSX**: application logic and declarative UI.
- **React DOM**: mounts each React application at the browser entry point.
- **React Router**:
  - `BrowserRouter` in the citizen app and TrackStaff.
  - `HashRouter` in the admin dashboard, which is useful for static hosting because navigation after deployment does not require server rewrite rules.
- **Create React App / `react-scripts`**: build and development toolchain for the citizen and admin applications.
- **Vite**: faster build and development toolchain for TrackStaff.

Backend and data access

- **Supabase JavaScript client**: used from the browser for authentication, database queries, database writes, file storage, RPC calls, and realtime updates.
- **Supabase Auth**: email/password registration, login, logout, current-user lookup, and password/session handling.
- **Supabase Postgres**: stores users/profiles, reports, categories, comments, likes, assignments, events, and resolution information.
- **Supabase Storage**: stores report photos, community media, advertisement media, and profile pictures.
- **Supabase Realtime**: the admin map subscribes to changes in the `reports` table and refreshes when reports are added or updated.

Visual and interaction libraries

- **React Leaflet + Leaflet**: interactive OpenStreetMap/Esri maps, markers, popups, circles, polygons, polylines, layer switching, and location selection.
- **Recharts**: pie, bar, donut, and trend visualisations.
- **Lucide React**: consistent SVG icons for navigation, statuses, categories, forms, alerts, and actions.
- **OpenStreetMap and Esri tile services**: map and satellite basemaps.
- **CSS and inline style objects**: responsive layouts, cards, badges, forms, modals, colour systems, and responsive breakpoints.
- **Testing Library/Jest setup**: available in the CRA applications for component and DOM testing.

3. Application startup and shared patterns

Citizen entry point

- `trackserv/src/index.js` renders `<App />` using `ReactDOM.createRoot`.
- `trackserv/src/App.js` defines the router and the authenticated dashboard shell.
- `BrowserRouter basename="/TrackServ"` supports the deployed GitHub Pages path.

Admin entry point

- `admin/src/index.js` renders the admin `<App />`.
- `admin/src/App.js` uses `HashRouter`, routes login and dashboard pages, and provides the shared admin shell.

Staff entry point

- `TrackStaff/servers/src/main.jsx` renders `<App />`.
- `TrackStaff/servers/src/App.jsx` uses `BrowserRouter basename="/TrackStaff/"`.

Shared implementation patterns

- Pages use `useState` for form values, filters, loading flags, modal state, selected records, and errors.
- `useEffect` loads Supabase data and cleans up subscriptions or timers.
- `useMemo` derives filtered rows, chart data, category summaries, and display values without unnecessary recalculation.
- `useCallback` stabilises handlers passed to child components.
- Loading and error states are explicitly represented in the UI.
- Small local components such as cards, buttons, badges, stat cards, chart components, empty states, and modals keep large pages organised.
- Five minutes of inactivity triggers logout in the citizen and admin dashboard shells. Activity is detected from mouse, keyboard, scroll, and touch events.

4. Citizen application: routes and modules

Route map

Defined in `trackserv/src/App.js`:

Route	Page/module	Purpose
`/`	`Landing`	Public product introduction and service overview
`/login`	`UserLogin`	Citizen authentication
`/register`	`Register`	Citizen account creation
`/home/*`	`Dashboard` shell	Authenticated citizen navigation
`/home`	`Home`	Citizen dashboard
`/home/reports`	`ReportsPage`	Browse and inspect community reports
`/home/reportIssues`	`ReportIssues`	Submit a municipal issue
`/home/about`	`About`	Explain TrackServ and show platform statistics
`/home/profile`	`Profile`	Manage citizen profile and view impact
`/home/community`	`CommunityPage`	Community posts and interactions

`AdvertisementsPage` is implemented as a citizen feature module and is available from the navigation/page code, although it is not selected by the current `Dashboard` switch in `App.js`. This is

worth mentioning as an available module rather than presenting it as a currently routed page.

Public landing page - `Body/Landing.jsx`

Frontend modules used

- React state/effects for responsive menu state and landing-page interactions.
- React Router `Link` and `useNavigate` for Login/Register calls to action.
- React Leaflet `MapContainer`, `TileLayer`, `Marker`, and `Tooltip` for a public service-location/report map.
- Leaflet for custom coloured marker icons.
- Lucide icons such as `FilePlus`, `CheckCircle2`, `Clock`, `BadgeCheck`, `X`, and `Menu`.
- Shared `Logo` component.

Presentation points

- This is the public marketing and onboarding surface.
- It explains the reporting workflow, shows trust/status indicators, and presents a map-based view of municipal activity.
- The page uses responsive navigation and a modal/section structure rather than requiring authentication for the first product explanation.

Citizen login - `Components/UserLogin.jsx`

- Uses React state for email, password, validation errors, authentication errors, password visibility, and submit progress.
- Uses React Router `Link`, `useLocation`, and `useNavigate`.
- Calls `supabase.auth.signInWithPassword`.
- After authentication, queries `profiles` and verifies that the profile role is `citizen`; non-citizen accounts are signed out and shown an explicit error.
- Includes accessible labels, `aria-invalid`, `aria-describedby`, autocomplete attributes, disabled submit state, and a show/hide password control.
- Redirects to `/home` after successful citizen authentication.

Citizen registration - `Components/Register.jsx`

- Controlled form for full name, email, password, and password confirmation.
- Performs client-side email and password consistency validation.
- Calls `supabase.auth.signUp` and passes `full_name` as user metadata.
- If a session is immediately available, upserts a citizen profile into `profiles`, including a default Tshwane location.
- If email confirmation is required, routes to login with a confirmation message.
- Uses the shared `Logo`, React Router links, and Supabase Auth.

Dashboard shell - `App.js` and `Components/Sidebar.jsx`

- `Dashboard` reads the URL with `useLocation` and chooses the active content module.
- `useNavigate` changes between child pages while preserving the shared shell.
- `Sidebar` provides the persistent navigation, category selection, user identity, profile access, and logout action.
- The shell owns the selected report category and passes it into the report/list pages.
- `useIdleLogout` signs out inactive users after five minutes and navigates to login.
- `CookieConsent` is mounted above the route tree so the consent experience is available throughout the application.

Citizen home - `Body/home.jsx`

Modules

- React hooks for loading, filtering, derived statistics, and callbacks.

- Supabase queries for reports and the current authenticated user.
- Supabase RPC `toggle_report_vote` for report voting.
- React Leaflet map with report markers, tooltips, and OpenStreetMap tiles.
- Recharts `PieChart`, `Pie`, and `Cell` for category/status summaries.
- Lucide icons for issue categories, actions, status, and calls to action.
- Shared `Footer`.

Functionality

- Loads recent/public reports and identifies the current user's voting state.
- Shows report status, severity, category, relative time, and vote controls.
- Provides a map view of geographically located reports.
- Provides category filtering and a visual category distribution.
- Links to report submission and the community.
- Uses helper functions such as `timeAgo`, `statusLabel`, `SeverityBadge`, and reusable card components.

Reports page - `Body/ReportsPage.jsx`

- Loads reports from `reports`, including category, reporter, votes, comments, and image-related information where available.
- Uses React state/effects/memoisation for search/filter state, pagination, loading, and the selected report modal.
- Displays summary/stat cards, category filtering, paginated report rows, status/severity indicators, and a detailed report view.
- Uses Lucide icons and the shared footer.
- `paginationItems`, `getCategoryVisual`, and `formatDate` keep display logic separate from the page layout.
- The page can route the user to `ReportIssues` through its report action.

Report an issue - `Body/ReportIssues.jsx`

Modules

- React hooks for controlled form state, file selection, validation, asynchronous submission, and map interactions.
- Supabase Auth `getUser` to associate the report with the citizen.
- Supabase queries to load `categories`.
- Supabase insert into `reports`.
- Supabase Storage `images` bucket for uploaded photos.
- Insert into `report_images` to associate uploaded media with the report.
- React Leaflet map for choosing/confirming latitude and longitude.
- Leaflet for marker handling and map resizing.
- Lucide icons for issue types, upload, location, tips, and validation.
- Local geometry helpers (`pointInRing`, `pointInPolygonCoords`) to constrain/validate the selected location.

User flow

1. Select a municipal service category.
2. Enter title/description and severity.
3. Choose a location from the map or location controls.
4. Attach supporting photos.
5. Submit the report to Supabase.
6. Upload files and create report-image records.
7. Show success or an explicit error while retaining useful form state.

About page - `Body/About.jsx`

- Uses parallel Supabase count queries to calculate platform figures:
  - total reports,
  - resolved/closed reports,
  - citizen accounts,
  - service categories.
- Uses React state/effects and Lucide icons.
- Presents the product mission, how the platform works, and live credibility statistics.
- Uses the shared footer.

Citizen profile - `Body/Profile.jsx`

- Loads and updates `profiles` and the user's reports.
- Uses Supabase Storage bucket `Profile` for profile-picture upload and public URL retrieval.
- Uses React state for tabs, edit mode, settings, upload state, chart filters, and error handling.
- Uses Recharts `ResponsiveContainer` and `BarChart` to show impact/report activity by time and category.
- Uses reusable local `Btn`, `Toggle`, and report-row components.
- Shows personal information, report history, status badges, account settings, notification/public-profile toggles, and impact statistics.

Community page - `Body/CommunityPage.jsx`

- Loads categories, likes, saved posts, comments, and the current user from Supabase.
- Supports feed filtering, post composition, comments, likes, saved posts, deletion, and structured post content.
- Uses Supabase Storage bucket `images` for post image/video media.
- Uses local reusable components such as `Chip`, `SidebarCard`, `Avatar`, and `CommentSection`.
- Uses React refs for file inputs and effects/memoisation for feed state and derived interaction state.
- Uses Lucide icons for social actions, media, navigation, and feedback.

Advertisements page - `Body/AdvertisementsPage.jsx`

- Loads service categories from `service_categories`.
- Provides search/filterable service listings and an “Add New Listing” flow.
- Uses Supabase Auth to identify the advertiser.
- Uploads listing media to the `images` bucket and obtains public URLs.
- Inserts payment records into `ad_payments`.
- Uses multi-step modal state for listing details, media upload, payment fields, processing, and success.
- Uses Lucide icons, responsive layout helpers, form controls, and the shared footer.

Citizen shared components

Component	Responsibility
`Logo.jsx`	Reusable TrackServ brand mark using a Lucide map-pin icon
`Sidebar.jsx`	Authenticated navigation, category filter, profile identity, logout
`footer.jsx`	Contact information, navigation, terms PDF, and footer interactions
`CookieConsent.jsx`	Persists and displays cookie-consent state
`supabaseClient.js`	Creates the shared Supabase browser client

5. Admin dashboard: routes and modules

Route map

Defined in `admin/src/App.js`:

Route	Page/module	Purpose
`/`	`AdminLogin`	Administrator authentication
`/dashboard/*`	`Dashboard` shell	Shared admin navigation and session handling
`/dashboard`	`AdminLocations`	Geospatial operational overview
`/dashboard/reports`	`ReportsPage`	Report list, filters, and details
`/dashboard/assignments`	`AssignmentsPage`	Assign reports to staff and track work
`/dashboard/analysis`	`AnalysisPage`	Operational analytics

Admin login - `pages/AdminLogin.jsx`

- Controlled email/password form with client-side validation.
- Uses `supabase.auth.signInWithPassword`.
- Reads the authenticated profile and checks for an administrative role.
- Signs out and reports an explicit error when the account is not authorised.
- Uses React Router navigation and Lucide icons.

Admin shell - `App.js` and `components/Sidebar.jsx`

- Uses `HashRouter` for static hosting compatibility.
- Selects the active page from the URL.
- Owns the selected category filter and passes it to report/assignment modules.
- Provides navigation for Locations, Reports, Assignments, and Analysis.
- Reads the logged-in administrator profile using Supabase Auth and `profiles`.
- Implements five-minute idle logout and explicit logout.
- Shared admin footer and sidebar keep the dashboard visually consistent.

Locations overview - `pages/AdminLocations.jsx`

- Queries `reports` with category, reporter, status, severity, coordinates, and resolution attendance data.
- Queries `categories` for filter and create-report controls.
- Subscribes to Supabase Realtime changes on `reports`, so the map refreshes when reports change elsewhere.
- Uses React Leaflet `MapContainer`, `TileLayer`, `CircleMarker`, and `Popup`.
- Supports Map and Satellite basemaps using OpenStreetMap and Esri tiles.
- Filters by category, status, and search text.
- Calculates overdue open work using a seven-day threshold.
- Hides resolved/closed markers after a configurable two-day retention period following attendance.
- Displays stat cards and a conic-gradient progress donut.
- Includes an admin modal for adding a location/report directly from the map.

Admin reports - `pages/ReportsPage.jsx`

- Loads reports, categories, and profile/assignee information from Supabase.
- Provides summary cards and a category donut chart.
- Supports category selection, pagination, status/severity presentation, report photos, and a detailed report modal.
- Uses helper components `Pill`, `Btn`, `IconBtn`, `StatCard`, `PhotoThumb`, and `ReportViewModal`.

- Uses memoisation for filtered/aggregated rows and callbacks for report actions.

Assignments - `pages/AssignmentsPage.jsx`

- Loads reports, staff profiles, events, and related report information.
- Supports staff selection, report search, assignment creation, assignment updates, and report detail inspection.
- Writes assignment/activity records to `events` and updates report assignment/status fields.
- Uses `images` storage for assignment/resolution evidence where applicable.
- Uses local `NewAssignmentRow`, `NewAssignmentModal`, `Overlay`, `Toast`, `EmptyState`, and photo thumbnail components.
- Provides user feedback through loading, error, empty, toast, and success states.

Analysis - `pages/AnalysisPage.jsx`

- Queries current-period, previous-period, and six-month report data from `reports`.
- Queries `categories` to build category-level analytics.
- Calculates overdue work, totals, percentage changes, status distribution, category distribution, trends, and workload.
- Uses local `StatCard`, `DonutChart`, `TrendChart`, `WorkloadBar`, `Select`, and `Card` components.
- Uses CSS/conic-gradient and SVG/chart primitives rather than relying on a large charting dependency in this page.
- Gives administrators a management view of volume, resolution performance, backlog, and category workload.

Admin shared components

Component	Responsibility
`components/Sidebar.jsx`	Role-aware navigation, profile identity, category filter, logout
`components/footer.jsx`	Contact details, terms PDF, and footer links
`components/supabaseClient.js`	Admin Supabase browser client

6. TrackStaff portal: routes and modules

Route map

Defined in `TrackStaff/servers/src/App.jsx`:

Route	Page/module	Purpose
`/login`	`pages/Login.jsx`	Staff authentication
`/dashboard`	`pages/Dashboard.jsx`	Assigned-work operational dashboard
`/profile`	`pages/Profile.jsx`	Staff profile and work history

Staff login - `pages/Login.jsx`

- Uses React state/effects for credentials, loading, errors, and password visibility.
- Uses Supabase Auth password login.
- Verifies that the authenticated account has an appropriate staff role; invalid roles are signed out.
- Uses Lucide `MapPin`, `Eye`, and `EyeOff`.
- Navigates to the staff dashboard after successful login.

Staff dashboard - `pages/Dashboard.jsx`

Map and data modules

- React state/effects/memoisation for reports, selected issue, filters, user location, route coordinates, resolutions, and modal state.
- Supabase queries for assigned reports, issue resolutions, current user, report events, and related profiles.
- Supabase updates to change report status to `in_progress` or return it to `open`.
- React Leaflet `MapContainer`, `TileLayer`, `Marker`, `Popup`, `Circle`, `Polygon`, and `Polyline`.
- Leaflet custom status markers and map bounds.
- Recharts for staff workload/status visualisations.
- Lucide icons for water, roads, safety, attendance, files, camera, comments, navigation, and completion actions.

Operational workflow

1. See assigned and nearby reports on the map.
2. Filter by all, resolved, in-progress, under-review, or unresolved status.
3. Select a report and inspect its description, priority, location, and history.
4. Start attendance when arriving at the site.
5. Continue work or mark the issue completed/rejected.
6. Upload a resolution photo and add a completion comment.
7. Store the resulting status/event/resolution data in Supabase.

The map is constrained to the Tshwane boundary, supports street/satellite layers, displays status-specific markers, and can show route/location context. Resolved items have a short map-retention period so the operational map stays focused on active work.

Staff profile - `pages/Profile.jsx`

- Uses Supabase to load staff profile data, reports, assigned reports, and resolutions.
- Uses Supabase Storage bucket `Profile` for profile-picture upload.
- Uses Recharts `ResponsiveContainer` and `BarChart` for activity/impact history.
- Uses local `Btn`, `Toggle`, and `ReportRow` components.
- Provides overview, activities, and settings tabs with editable contact/profile information, notification controls, report history, and work statistics.

Staff location/admin asset - `assets/AdminLocations.jsx`

- A map-oriented operational component also exists under `TrackStaff/servers/src/assets`.
- It uses React Leaflet, report/category Supabase queries, status metadata, overdue calculations, stat cards, and report creation controls.
- It follows the same map/status language as the admin dashboard and can be treated as reusable operational map functionality.

Staff shared components

Component	Responsibility
`components/Sidebar.jsx`	Staff navigation, role display, category filtering, logout
`components/footer.jsx`	Shared contact/terms footer
`components/supabaseClient.js`	Staff Supabase browser client

7. Shared Supabase data model used by the frontend

Supabase resource	Frontend usage
`auth.users` / Supabase Auth	Registration, login, logout, current-user lookup
`profiles`	Name, username, role, location, profile picture, staff/admin/citizen identity
`categories`	Report categories, category labels, filters, map visuals
`service_categories`	Advertisement/service-listing categories
`reports`	Core citizen issue records, location, status, severity, assignment, timestamps
`report_images`	Links uploaded report photos to reports
`report_votes`	Citizen voting on reports
`likes`	Community post likes
`saved_posts`	Community post bookmarks
`comments`	Community post comments
`events`	Assignment and operational history
`issue_resolutions`	Staff attendance/resolution information
`ad_payments`	Advertisement listing payment records
`images` storage bucket	Report photos, community media, advertisement media
`Profile` storage bucket	User/staff profile pictures
`toggle_report_vote` RPC	Atomic report-vote toggle operation

8. End-to-end product flow for the presentation

Use this sequence as the main live/demo narrative:

1. **Discover**: Start at the public landing page and explain the civic-service problem.
2. **Join**: Register a citizen account through Supabase Auth and create the related `profiles` row.
3. **Report**: Submit an issue with category, description, severity, map location, and photo.
4. **Track**: Return to the citizen home/reports pages to see status, votes, map position, and report details.
5. **Manage**: Log into the admin dashboard to see the report appear on the live locations map and in the reports list.
6. **Assign**: Use Assignments to select a staff member and create the operational event.
7. **Attend**: Log into TrackStaff, open the assigned issue, start attendance, update progress, upload evidence, and complete the work.
8. **Measure**: Show the admin Analysis page and citizen Profile impact chart reflecting the service activity.
9. **Engage**: Show Community for resident discussion and Advertisements for the wider service ecosystem.

9. Strengths to highlight

- **Role separation**: citizens, administrators, and field staff each get focused workflows.
- **Single source of truth**: all frontends use the same Supabase data and authentication model.
- **Geospatial UX**: the report lifecycle is visible on maps, not only in tables.
- **Realtime operations**: admin locations can refresh when report data changes.
- **Evidence-based resolution**: staff can attach images and comments to completed work.
- **Data-driven decisions**: dashboards convert report data into status, trend, category, and workload analytics.

- **Reusable UI language**: shared sidebars, footers, icons, cards, badges, and colour/status conventions reduce cognitive load.
- **Responsive design**: pages use viewport-aware state and responsive layouts for desktop and smaller screens.

10. Honest implementation notes and future improvements

- The two CRA apps and the Vite app currently have separate Supabase client files and duplicated UI patterns. A shared package/design system could reduce duplication.
- Authentication is enforced in page logic and role checks; production deployments should also rely on Supabase Row Level Security policies so browser clients cannot bypass role restrictions.
- The current admin/citizen shells use an inactivity timer, but a future version could centralise session/auth context and handle auth state changes in one place.
- Some pages use extensive inline styling while others use CSS files. A design-token system or component library would make colour, spacing, and responsive behaviour easier to maintain.
- The admin dashboard uses a hash router for static deployment, while the citizen and staff applications use browser history routing. Deployment configuration should keep these routing assumptions explicit.

11. Suggested presentation structure

1. Problem and product goal.
2. Three-user architecture diagram: citizen → admin → field staff.
3. Technology stack.
4. Routing and application shells.
5. Citizen pages and report-submission workflow.
6. Admin pages and operational analytics.
7. TrackStaff pages and field-resolution workflow.
8. Supabase data/storage/realtime integration.
9. Maps, charts, responsive UI, and reusable components.
10. End-to-end demonstration and future improvements.